Analiza imaginilor satelitare folosind CNN și U-Net pe datasetul EuroSAT

Proiect realizat în Python folosind PyTorch

Cuprins

1. Introducere
2. Obiectivele proiectului
3. Tehnologii utilizate
4. Datasetul EuroSAT
5. Conceptul de Deep Learning
6. Rețele neuronale convoluționale (CNN)
7. Arhitectura CNN utilizată
8. Procesul de antrenare
9. Metrici de evaluare
10. Matricea de confuzie
11. Modelul U-Net
12. Segmentarea imaginilor
13. Rezultatele obținute
14. Explicații pentru cod
15. Probleme întâlnite și soluții
16. Concluzii
17. Bibliografie

---

1. Introducere

În ultimii ani, inteligența artificială și tehnicile de Deep Learning au devenit extrem de importante în analiza imaginilor. Un domeniu foarte important este analiza imaginilor satelitare, folosită în agricultură, monitorizarea mediului, urbanism și detectarea schimbărilor climatice.

În cadrul acestui proiect am realizat un sistem capabil să analizeze imagini satelitare folosind două modele de rețele neuronale:

- CNN (Convolutional Neural Network) pentru clasificarea imaginilor
- U-Net pentru segmentarea imaginilor

Datasetul utilizat este EuroSAT, un set de imagini satelitare RGB împărțite în 10 categorii diferite.

Scopul principal al proiectului a fost înțelegerea modului în care funcționează rețelele neuronale convoluționale și aplicarea lor într-un scenariu real.

---

2. Obiectivele proiectului

Principalele obiective ale proiectului sunt:

- clasificarea imaginilor satelitare în 10 clase;
- implementarea unui model CNN de la zero;
- implementarea unui model U-Net pentru segmentare;
- antrenarea modelelor pe datasetul EuroSAT;
- evaluarea performanței folosind metrici specifice;
- generarea unei matrici de confuzie;
- salvarea automată a graficelor și rezultatelor.

---

3. Tehnologii utilizate

În realizarea proiectului au fost folosite următoarele tehnologii:

Tehnologie	Rol
Python	Limbaj principal
PyTorch	Deep Learning
NumPy	Operații matematice
Matplotlib	Grafice și imagini
Scikit-Learn	Metrici și matrice de confuzie
Seaborn	Vizualizarea matricii de confuzie
PIL	Procesarea imaginilor

---

4. Datasetul EuroSAT

Datasetul EuroSAT conține imagini satelitare RGB cu dimensiunea 64x64 pixeli.

Acesta conține aproximativ 27.000 de imagini împărțite în 10 clase:

- AnnualCrop
- Forest
- HerbaceousVegetation
- Highway
- Industrial
- Pasture
- PermanentCrop
- Residential
- River
- SeaLake

Datasetul este foarte utilizat în proiecte de clasificare a imaginilor satelitare.

Exemple de imagini:

- păduri;
- râuri;
- lacuri;
- drumuri;
- zone industriale;
- zone rezidențiale.

---

5. Conceptul de Deep Learning

Deep Learning reprezintă un subdomeniu al inteligenței artificiale bazat pe rețele neuronale artificiale.

Aceste modele sunt inspirate de modul în care funcționează creierul uman și pot învăța automat caracteristici importante din date.

În cazul imaginilor, rețelele neuronale convoluționale pot detecta:

- margini;
- forme;
- texturi;
- obiecte complexe.

Avantajul major al Deep Learning este că modelul învață singur caracteristicile importante fără reguli definite manual.

---

6. Rețele neuronale convoluționale (CNN)

CNN (Convolutional Neural Network) este un tip special de rețea neuronală folosită pentru imagini.

Un CNN conține mai multe tipuri de straturi:

6.1 Straturi Convoluționale

Aceste straturi aplică filtre asupra imaginilor pentru a detecta caracteristici.

Exemple:

- margini;
- colțuri;
- forme;
- texturi.

6.2 Funcția de activare ReLU

ReLU introduce non-linearitate în model:

```
nn.ReLU(inplace=True)
```

6.3 MaxPooling

Reduce dimensiunea imaginilor și păstrează informațiile importante.

```
nn.MaxPool2d(2)
```

6.4 Fully Connected Layer

La final, caracteristicile extrase sunt transformate într-o predicție finală.

---

7. Arhitectura CNN utilizată

Modelul implementat în proiect conține:

- 4 straturi convoluționale;
- funcții ReLU;
- straturi MaxPooling;
- Dropout pentru reducerea overfittingului;
- două straturi fully connected.

Fragment din cod:

```
self.features = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2),

    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2),
)
```

Modelul primește imagini RGB și returnează probabilitatea pentru fiecare clasă.

---

8. Procesul de antrenare

Datele au fost împărțite în:

- 70% pentru antrenare;
- 15% pentru validare;
- 15% pentru testare.

În timpul antrenării:

1. imaginile sunt introduse în rețea;
2. modelul produce predicții;
3. se calculează eroarea;
4. se actualizează greutățile rețelei.

Pentru optimizare a fost folosit algoritmul Adam:

```
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Funcția de pierdere utilizată:

```
criterion = nn.CrossEntropyLoss()
```

---

9. Metrici de evaluare

Pentru evaluarea modelului au fost folosite mai multe metrici.

9.1 Accuracy

Reprezintă procentul total de predicții corecte.

Formula:

[Accuracy =]

Rezultatul obținut:

- aproximativ 92%

9.2 Precision

Măsoară cât de corecte sunt predicțiile pozitive.

9.3 Recall

Măsoară cât de bine găsește modelul exemplele corecte.

9.4 F1-Score

Reprezintă media armonică dintre precision și recall.

Cod utilizat:

```
accuracy = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds, average="macro")
recall = recall_score(all_labels, all_preds, average="macro")
f1 = f1_score(all_labels, all_preds, average="macro")
```

---

10. Matricea de confuzie

Matricea de confuzie este foarte importantă deoarece arată exact unde greșește modelul.

Aceasta compară:

- clasele reale;
- clasele prezise.

O matrice ideală are valori mari pe diagonală.

Fragment din cod:

```
cm = confusion_matrix(all_labels, all_preds)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
```

Imaginea matricii de confuzie este salvată automat în folderul:

`images/cnn_confusion_matrix.png`

---

11. Modelul U-Net

U-Net este o arhitectură specială pentru segmentare semantică.

Acest model este foarte folosit în:

- imagistică medicală;
- imagini satelitare;
- detectarea obiectelor.

Arhitectura U-Net are două componente principale:

Encoder

Extrage caracteristici importante și reduce dimensiunea imaginilor.

Decoder

Reconstruiește imaginea segmentată.

---

12. Segmentarea imaginilor

Datasetul EuroSAT nu conține măști reale de segmentare.

Din acest motiv, în proiect au fost create pseudo-măști.

Fiecare pixel primește eticheta imaginii.

Exemplu:

- dacă imaginea este „Forest”, atunci toți pixelii sunt etichetați ca „Forest”.

Acest lucru nu reprezintă segmentare semantică reală, dar permite demonstrarea funcționării modelului U-Net.

Fragment din cod:

```
mask = torch.full((IMAGE_SIZE, IMAGE_SIZE),
                  fill_value=label,
                  dtype=torch.long)
```

---

13. Rezultatele obținute

CNN

Rezultatele obținute pentru clasificare:

Metrică	Valoare
Accuracy	~92%
Precision	~92%
Recall	~92%
F1-Score	~92%

U-Net

Rezultatul pentru segmentare:

Metrică	Valoare
Pixel Accuracy	~83%

Modelul CNN a avut performanțe foarte bune pentru clasificarea imaginilor satelitare.

---

14. Explicații pentru cod

Detectarea dispozitivului

Codul detectează automat dacă există GPU.

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Transformarea imaginilor

```
transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
])
```

Salvarea modelelor

```
torch.save(model.state_dict(), "best_eurosat_cnn.pth")
```

Predicția unei imagini

```
pred = torch.argmax(logits, dim=1).item()
```

---

15. Probleme întâlnite și soluții

Problema 1 — Lipsa GPU

Modelul a fost antrenat pe CPU, ceea ce a crescut timpul de antrenare.

Soluție

Reducerea dimensiunii imaginilor și utilizarea batch size optim.

Problema 2 — Erori de indentare

Python este sensibil la indentare.

Soluție

Corectarea blocurilor de cod și mutarea funcțiilor în interiorul claselor sau funcțiilor potrivite.

Problema 3 — Matricea de confuzie

A fost necesară instalarea bibliotecilor:

```
pip install scikit-learn seaborn
```

---

16. Concluzii

Acest proiect demonstrează eficiența modelelor de Deep Learning în analiza imaginilor satelitare.

Modelul CNN a obținut rezultate foarte bune pentru clasificare, iar U-Net a demonstrat capacitatea de segmentare.

Prin acest proiect au fost înțelese:

- principiile rețelelor neuronale convoluționale;
- procesul de antrenare;
- evaluarea performanței;
- utilizarea metricilor;
- interpretarea matricii de confuzie.

Proiectul poate fi extins în viitor prin:

- folosirea unui GPU;
- utilizarea unor modele mai avansate;
- utilizarea segmentării reale;
- creșterea numărului de clase.

---

17. Bibliografie

1. PyTorch Documentation
2. Scikit-Learn Documentation
3. EuroSAT Dataset
4. Goodfellow — Deep Learning
5. Ian Goodfellow, Yoshua Bengio, Aaron Courville
6. U-Net: Convolutional Networks for Biomedical Image Segmentation
7. Documentația oficială NumPy și Matplotlib

Anexe

Exemplu de rulare

```
python eurosat_cnn_unet.py
```

Fișiere generate automat

- best_eurosat_cnn.pth
- best_eurosat_unet_demo.pth
- logs.txt
- images/EuroSAT_CNN_loss.png
- images/EuroSAT_CNN_score.png
- images/cnn_confusion_matrix.png
- images/unet_result_0.png

Concluzie finală

Proiectul reprezintă o aplicație practică de inteligență artificială în domeniul viziunii computerizate și demonstrează modul în care rețelele neuronale pot analiza și interpreta imagini satelitare cu o precizie ridicată.